

Содержание

Задание.....	3
Реализация запросов на SQL.....	4
Планы выполнения запросов.....	4
Ответы на вопросы.....	7
Вопрос 1.....	7
Вопрос 2.....	7
Вопрос 3.....	7
Вывод.....	8

Задание

Реализовать запросы на языке SQL:

1. Сделать запрос для получения атрибутов из указанных таблиц, применив фильтры по указанным условиям:
Таблицы: Н_ЛЮДИ, Н_ВЕДОМОСТИ.
Вывести атрибуты: Н_ЛЮДИ.ИД, Н_ВЕДОМОСТИ.ЧЛВК_ИД.
Фильтры (AND):
а) Н_ЛЮДИ.ОТЧЕСТВО > Владимирович.
б) Н_ВЕДОМОСТИ.ЧЛВК_ИД > 105590.
Вид соединения: INNER JOIN.
2. Сделать запрос для получения атрибутов из указанных таблиц, применив фильтры по указанным условиям:
Таблицы: Н_ЛЮДИ, Н_ВЕДОМОСТИ, Н_СЕССИЯ.
Вывести атрибуты: Н_ЛЮДИ.ОТЧЕСТВО, Н_ВЕДОМОСТИ.ДАТА, Н_СЕССИЯ.ЧЛВК_ИД.
Фильтры (AND):
а) Н_ЛЮДИ.ИМЯ > Александр.
б) Н_ВЕДОМОСТИ.ДАТА = 2010-06-18.
с) Н_СЕССИЯ.УЧГОД = 2003/2004.
Вид соединения: INNER JOIN.

Ответить на вопросы:

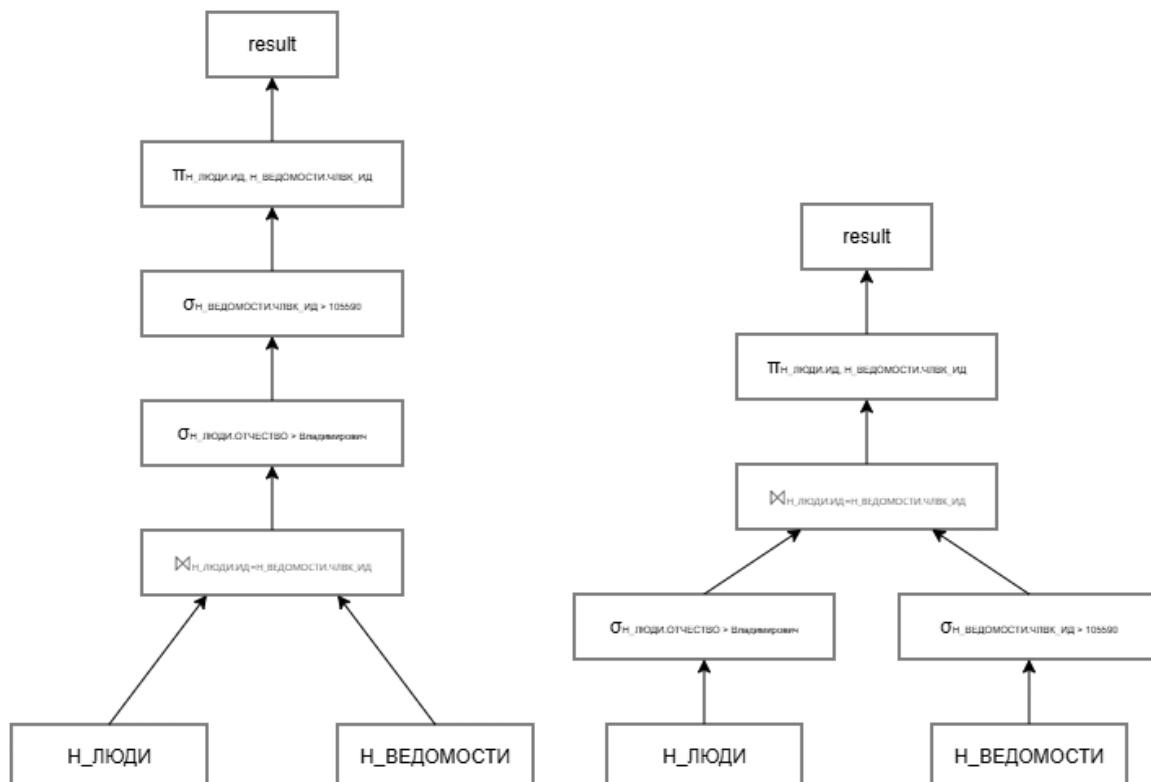
1. Для каждого запроса предложить индексы, добавление которых уменьшит время выполнения запроса (указать таблицы/атрибуты, для которых нужно добавить индексы, написать тип индекса; объяснить, почему добавление индекса будет полезным для данного запроса).
2. Для запросов 1-2 необходимо составить возможные планы выполнения запросов. Планы составляются на основании предположения, что в таблицах отсутствуют индексы. Из составленных планов необходимо выбрать оптимальный и объяснить свой выбор. Изменяются ли планы при добавлении индекса и как?
3. Для запросов 1-2 необходимо добавить в отчет вывод команды EXPLAIN ANALYZE [запрос].

Реализация запросов на SQL

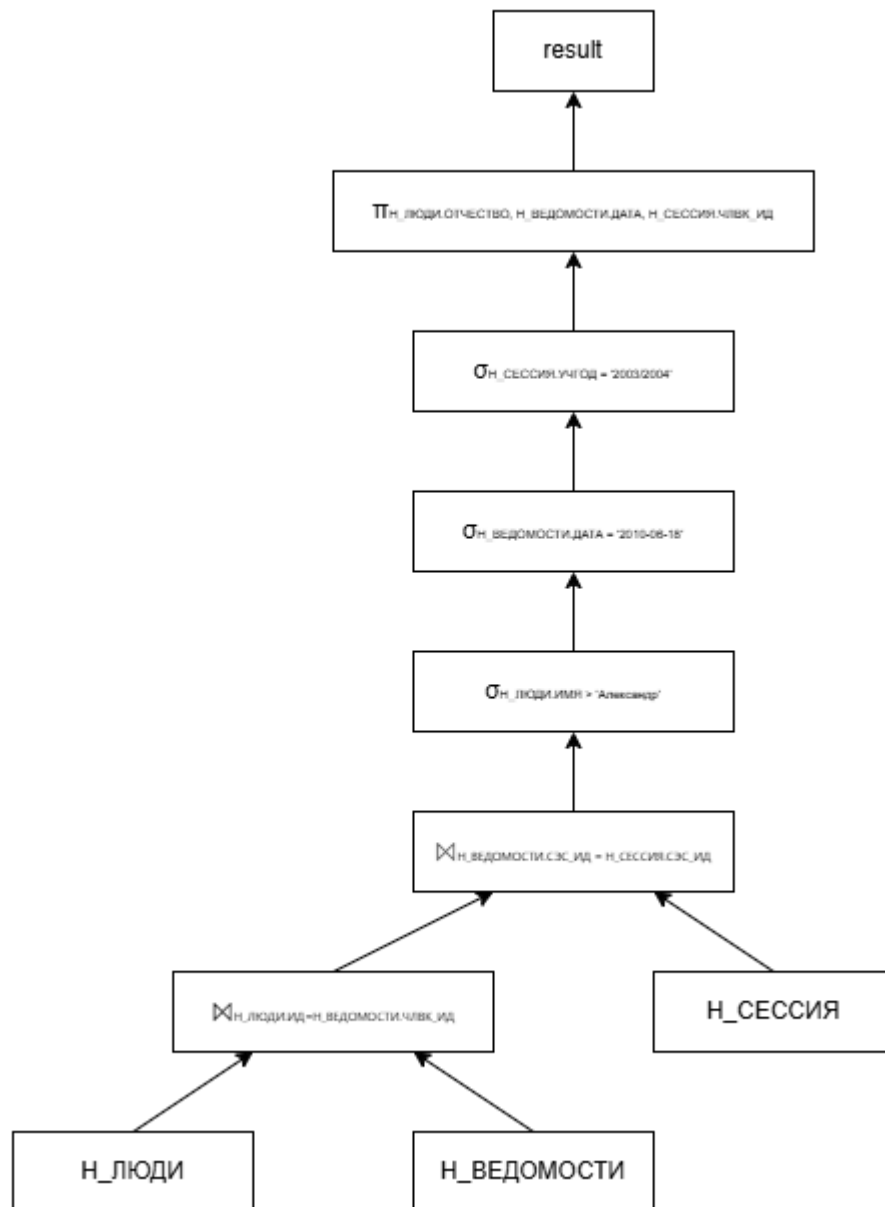
1. SELECT Н_ЛЮДИ.ИД, Н_ВЕДОМОСТИ.ЧЛВК_ИД FROM Н_ЛЮДИ
INNER JOIN Н_ВЕДОМОСТИ ON Н_ЛЮДИ.ИД =
Н_ВЕДОМОСТИ.ЧЛВК_ИД
WHERE Н_ЛЮДИ.ОТЧЕСТВО > 'Владимирович'
AND Н_ВЕДОМОСТИ.ЧЛВК_ИД > 105590;
2. SELECT Н_ЛЮДИ.ОТЧЕСТВО, Н_ВЕДОМОСТИ.ДАТА,
Н_СЕССИЯ.ЧЛВК_ИД FROM Н_ЛЮДИ
INNER JOIN Н_ВЕДОМОСТИ ON Н_ЛЮДИ.ИД =
Н_ВЕДОМОСТИ.ЧЛВК_ИД
INNER JOIN Н_СЕССИЯ ON Н_ВЕДОМОСТИ.СЭС_ИД =
Н_СЕССИЯ.СЭС_ИД
WHERE Н_ЛЮДИ.ИМЯ > 'Александр'
AND Н_ВЕДОМОСТИ.ДАТА = '2010-06-18'
AND Н_СЕССИЯ.УЧГОД = '2003/2004';

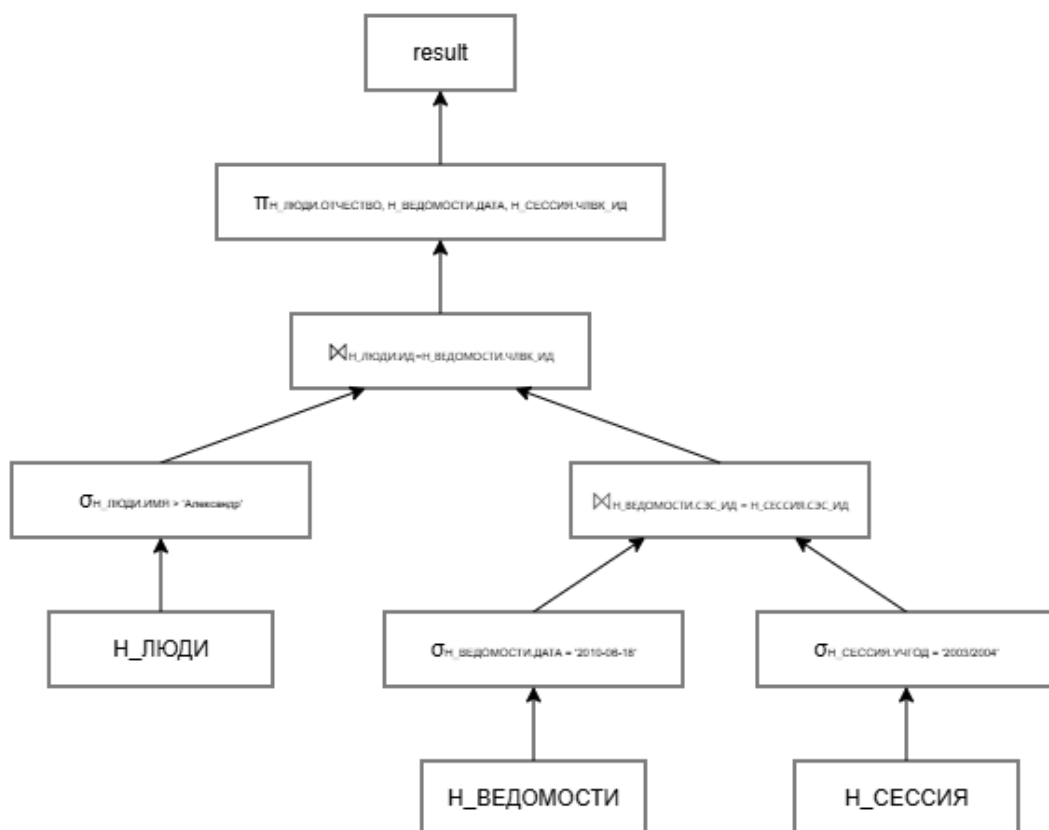
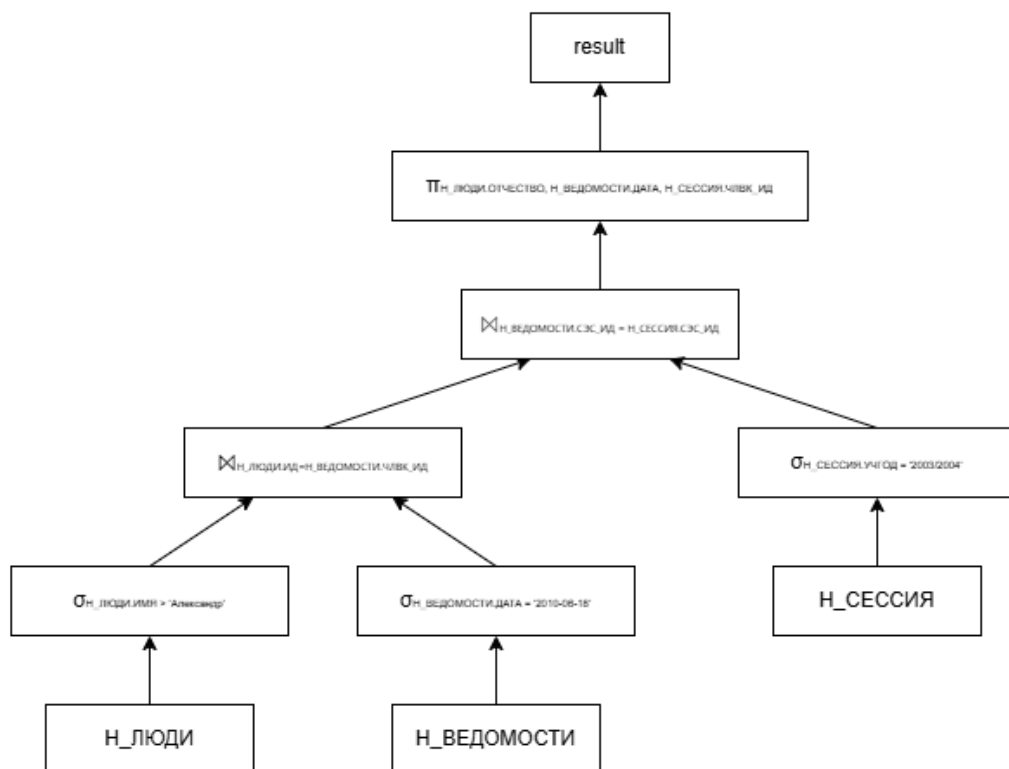
Планы выполнения запросов

Примеры планов для запроса 1:



Примеры планов для запроса 2:





Ответы на вопросы

Вопрос 1

В первом запросе и в соединении таблиц, и в блоке WHERE, присутствует атрибут ИД (или ЧЛВК_ИД). Части запроса с его участием можно оптимизировать, добавив индекс для этого атрибута. Так как в запросе проверяется условие со знаком ">", в данном случае тип индекса лучше всего поставить B-tree. Аналогично этому способу, можно оптимизировать запрос на выборку атрибута ОТЧЕСТВО, также используя B-tree.

Во втором запросе также присутствует склеивание таблиц по атрибуту ЧЛВК_ИД, поэтому стратегия из первого запроса также может помочь и во втором. Для атрибута ИМЯ это сработает также, ведь он аналогичен атрибуту из первого запроса. Так как Н_ВЕДОМОСТИ.ДАТА имеет много строк с одной и той же датой, при добавлении индекса в запрос из-за низкой селективности это не будет оптимальным решением в достаточно большом количестве случаев, поэтому добавлять индекс туда не стоит. То же самое можно сказать и про УЧГОД.

Вопрос 2

Составленные планы предложены в пункте выше.

Для первого запроса оптимальным планом является правый, так как в нем соединение таблиц происходит только после фильтрации данных в каждой из них, что снижает требуемое время для этой операции.

Для второго запроса оптимальным планом будет третий вариант, так как по результату первого соединения отсеется большая часть ненужных данных, а значит и второе соединение пройдет быстрее.

Добавление индекса в таблицу не сильно повлияет на план запроса. В некоторых случаях для планов изменится нахождение проекции (выделение из таблицы нужных столбцов), так как из-за специфики запроса может быть использована стратегия Index Only Scan. В остальном, сам план запроса сильно не поменяется.

Вопрос 3

Вывод для первого запроса:

```
Hash Join (cost=196.06..5577.79 rows=110492 width=8) (actual time=3.718..57.737
rows=108933 loops=1)
  Hash Cond: ("Н_ВЕДОМОСТИ"."ЧЛВК_ИД" = "Н_ЛЮДИ"."ИД")
    ->    Index Only Scan using "ВЕД_ЧЛВК_FK_IFK" on "Н_ВЕДОМОСТИ"
(cost=0.29..4797.81 rows=222375 width=4) (actual time=0.010..23.148 rows=222413
loops=1)
```

```

      Index Cond: ("ЧЛВК_ИД" > 105590)
      Heap Fetches: 0
    -> Hash (cost=163.97..163.97 rows=2543 width=4) (actual time=3.655..3.656
rows=2544 loops=1)
      Buckets: 4096 Batches: 1 Memory Usage: 122kB
    -> Seq Scan on "Н_ЛЮДИ" (cost=0.00..163.97 rows=2543 width=4) (actual
time=0.028..3.241 rows=2544 loops=1)
      Filter: (("ОТЧЕСТВО")::text > 'Владимирович'::text)
      Rows Removed by Filter: 2574
Planning Time: 1.516 ms
Execution Time: 62.762 ms

```

Вывод для второго запроса:

```

Nested Loop (cost=122.35..355.83 rows=3 width=32) (actual time=1.013..1.015
rows=0 loops=1)
  -> Hash Join (cost=122.07..339.27 rows=4 width=16) (actual time=1.012..1.014
rows=0 loops=1)
    Hash Cond: ("Н_ВЕДОМОСТИ"."СЭС_ИД" = "Н_СЕССИЯ"."СЭС_ИД")
    -> Index Scan using "ВЕД_ДАТА_I" on "Н_ВЕДОМОСТИ"
(cost=0.29..216.82 rows=73 width=16) (actual time=0.041..0.139 rows=141 loops=1)
      Index Cond: ("ДАТА" = '2010-06-18 00:00:00'::timestamp without
time zone)
    -> Hash (cost=117.90..117.90 rows=310 width=8) (actual
time=0.824..0.825 rows=310 loops=1)
      Buckets: 1024 Batches: 1 Memory Usage: 20kB
    -> Seq Scan on "Н_СЕССИЯ" (cost=0.00..117.90 rows=310 width=8)
(actual time=0.027..0.766 rows=310 loops=1)
      Filter: (("УЧГОД")::text = '2003/2004'::text)
      Rows Removed by Filter: 3442
  -> Index Scan using "ЧЛВК_ПК" on "Н_ЛЮДИ" (cost=0.28..4.14 rows=1
width=24) (never executed)
    Index Cond: ("ИД" = "Н_ВЕДОМОСТИ"."ЧЛВК_ИД")
    Filter: (("ИМЯ")::text > 'Александр'::text)
Planning Time: 2.007 ms
Execution Time: 1.103 ms

```

Вывод

Во время данной лабораторной работы были изучены использование индексов в реляционных базах данных, составление плана выполнения запроса и его оптимизация, а также практическое применение данных знаний с помощью СУБД PostgreSQL.